

TECH

REACT ARCHITECTURE

REACT ARCHITECTURE

REACT ARCHITECTURE

35 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК [feature-sliced.design](#) / [react.dev](#) / общая практика

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	СТРУКТУРА / FEATURES / PAGES Q1 · Q2 · Q3 · Q4 · Q5	FSD SHARED / ENTITIES	База: как разрезать большое React-приложение на слои.
ДЕНЬ 2 СРЕДНИЙ	ГДЕ ЖИТЬ ESIGN SYSTEM Q6 · Q7 · Q8 · Q9	HOOKS DOMAIN UI	D Куда складывать хуки, сервисы, компоненты, и зачем DS.
ДЕНЬ 3 ТЯЖЁЛЫЙ	DOMAIN ↔ UI TABLE Q10 · Q11 · Q12	DATA FETCHING ↔ RENDER	Главный навык — отделять предметную логику от рендера.
ДЕНЬ 4 СРЕДНИЙ	FORM /EMPTY/ERROR Q13 · Q14 · Q15 · Q21	WIZARD DASHBOARD LOADING	Типовые UI-паттерны и как не превратить их в ад.
ДЕНЬ 5 ТЯЖЁЛЫЙ	ROLES ERROR BOUNDARIES Q16 · Q17 · Q18 · Q19 · Q20	PERMISSIONS UNAUTHORIZED	Безопасность UI и устойчивость к ошибкам.
ДЕНЬ 6 СРЕДНИЙ	DEGRADATION n THEMING Q22 · Q23 · Q24 · Q25 · Q26	FLAGS ROLLOUT	i18n Релизы по чуть, языки и темы.
ДЕНЬ 7 СРЕДНИЙ	RESPONSIVE REVIEW Q27 · Q28 · Q29 · Q30 · Q31 · Q32 · Q33 · Q34 · Q35	SHARED API ADR DRILLING	Качество кода и процесс принятия решений.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 5 ВОПРОСОВ

СТРУКТУРА FSD SHARED / ENTITIES / FEATURES / PAGES**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

База: как разрезать большое React-приложение на слои.

ВОПРОСЫ ДНЯ

- Q01 Как ты структурируешь большое React-приложение?
- Q02 Что думаешь про Feature-Sliced Design?
- Q03 Как разделяешь shared, entities, features, pages?
- Q04 Как не превратить shared в помойку?
- Q05 Где должны жить API clients?

Как ты структурируешь большое React-приложение?

ОПИСАНИЕ

Универсального ответа нет, но эмпирика проверена: разрезать по слоям и feature-границам. Самые частые подходы – Feature-Sliced Design (русскоязычный стандарт), Clean Architecture-инспирация, Domain-Driven design-light. Главное – явная иерархия слоёв (от низшего к высшему), запрет на обратные импорты и единые правила, где жить API-клиентам, хукам, UI-компонентам.

КАК РАБОТАЕТ

- 01 Слои сверху вниз: app (entrypoint) → pages → widgets → features → entities → shared.
- 02 Импорт идёт ТОЛЬКО вниз: pages могут импортировать из всех ниже, shared – ни из кого.
- 03 Внутри слоя – модули по доменной/функциональной границе.
- 04 API-клиенты, хуки, UI и domain-логика разнесены по типам.
- 05 Линтером (eslint-plugin-boundaries / @feature-sliced/steiger) – проверка правил.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой проект на 50+ компонентов.
- 02 Команда из 3+ разработчиков.
- 03 Долгоживущий продукт.

ПОДВОДНЫЕ КАМНИ

- 01 Без линтера правила нарушаются за пару спринтов.
- 02 Слишком много слоёв – over-engineering.
- 03 Папка shared без правил → помойка (см. Q4).
- 04 Сильная связь features → невозможно вынести.

КОД

// Скелет FSD

```
src/
├─ app/      # entry, providers, store-wiring
├─ pages/    # маршруты
├─ widgets/  # композиции features (Header, Sidebar)
├─ features/ # пользовательские действия (login, logout)
├─ entities/ # доменные сущности (User, Post)
└─ shared/   # ui-kit, hooks, lib, api
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем FSD отличается от Atomic Design?

02 Почему shared нельзя импортировать из features?

03 Какие альтернативы FSD?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/>

Что думаешь про Feature-Sliced Design?

ОПИСАНИЕ

FSD – методология структурирования frontend: слои + срезы (по доменным feature) + сегменты (api/lib/ui/model/config). Хорошо подходит крупным проектам с множеством разработчиков и долгим жизненным циклом. Минусы: кривая обучения, избыточность для маленьких проектов, иногда конфликтует с фреймворками (Next App Router).

КАК РАБОТАЕТ

- 01 Слои: app, pages, widgets, features, entities, shared.
- 02 Слайсы внутри слоя – по бизнес-смыслу.
- 03 Сегменты внутри слайса: api, lib, ui, model, config.
- 04 Импорт строго сверху вниз и горизонтально не допускается.
- 05 Public API через index.ts – внутреннее не утекает.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Средние и крупные SPA.
- 02 Несколько команд с общим кодом.
- 03 Долгоживущие продукты.
- 04 Проекты, где быстрый onboarding критичен.

ПОДВОДНЫЕ КАМНИ

- 01 Лишние слои в маленьком проекте.
- 02 Конфликт с конвенциями Next App Router (page-based) – нужны компромиссы.
- 03 Public-API через index.ts увеличивает bundle при неправильной tree-shake-настройке.
- 04 Без линтера / документации команда забывает правила.

КОД

// Импорт по правилам

```
// pages/HomePage.tsx
import { Header } from 'widgets/Header';
import { LoginButton } from 'features/auth';
import { UserCard } from 'entities/user';
import { Button } from 'shared/ui';
```

МОГУТ ДОСПРОСИТЬ

- 01 Как сочетать FSD и Next App Router?
- 02 Что такое Public API в FSD?
- 03 Когда FSD не нужен?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs>

Как разделяешь shared, entities, features, pages?

ОПИСАНИЕ

Принципы: shared – переиспользуемые примитивы без доменной логики; entities – доменные сущности (User, Post, Order) с собственными api/model/ui; features – пользовательские действия (login, addToCart, publishPost), которые могут менять одну или несколько сущностей; pages – композиция features и widgets под конкретный маршрут.

КАК РАБОТАЕТ

- 01 shared – Button, Modal, useDebounce, fetchClient, format-utils.
- 02 entities – UserCard, useUser, userApi (про сущность, не про действие).
- 03 features – LoginForm, useLogout, addToCartButton (целевое действие).
- 04 pages – компонует widgets/features, привязывает к роуту.
- 05 widgets – большие композиции (Header, Sidebar) поверх features.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 При новой странице – что положить и куда.
- 02 При рефакторинге легаси.
- 03 Code review – проверять, не уехал ли код в shared без права.

ПОДВОДНЫЕ КАМНИ

- 01 Доменная логика в shared – нарушение слоя.
- 02 Cross-import между features → циклы.
- 03 Entity, у которой нет api или model – пустая папка.
- 04 Слишком крупная feature, делающая «всё» – кандидат на разбиение.

КОД

// Пример распределения

```
shared/ui/Button
entities/user/{api, model, ui/UserCard}
features/auth/login/{ui/LoginForm, model/useLogin}
pages/SignInPage
```

МОГУТ ДОСПРОСИТЬ

- 01 Куда поместить «UserCard с кнопкой добавить в чат»?
- 02 Что считать widget?
- 03 Когда entity делить на несколько?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs/get-started/overview>

ВОПРОС Q04

Как не превратить shared в помойку?

ОПИСАНИЕ

Правила: shared только для AGNOSTIC-кода (без знаний о домене), любой кусок shared должен быть переиспользован 2+ раза, любое доменное знание (название поля API, бизнес-правило) – это уже entity или feature. Линтер + code review + регулярная ревизия ("что в shared, чего там быть не должно").

КАК РАБОТАЕТ

- 01 Rule of three: код становится shared только при 3-м использовании.
- 02 В shared не импортируется ничего из верхних слоёв.
- 03 Shared делится по сегментам: ui, lib, hooks, api, config.
- 04 Регулярные «архитектурные дни» – ревизия shared.
- 05 Линтер @feature-sliced или eslint-plugin-boundaries.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 При создании нового хелпера – задавать вопрос «это правда универсально?».
- 02 Code review – главный фильтр.
- 03 Архитектурные обсуждения раз в месяц.

ПОДВОДНЫЕ КАМНИ

- 01 DateUtils, который форматирует «дату оплаты в формате магазина» – это уже не shared.
- 02 Shared/utils/index.ts с 200 экспортами – mark для рефакторинга.
- 03 shared/ui/Button с props.variant='premium' – это уже доменное знание.
- 04 Без линтера правила нарушаются.

КОД

```
// Что shared, что нет
```

```
// shared
shared/ui/Button      – безусловная UI-примитив
shared/lib/format-date – общий форматтер

// НЕ shared
shared/ui/PaymentBadge – знает про домен 'оплата'
shared/lib/calculateTax – бизнес-правила
```

МОГУТ ДОСПРОСИТЬ

- 01 Что делать с почти-shared, но с одной доменной фичей?
- 02 Где граница между "переиспользуем" и "копипаста ок"?

03 Как чистить shared от мусора?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs/reference/layers>

ВОПРОС Q05

Где должны жить API clients?

ОПИСАНИЕ

API-клиенты живут на уровне ENTITY (если относятся к конкретной сущности) или в `shared/api` (если универсальный fetch-клиент с `auth/retry`). Никогда не разбрасывать `fetch`-вызовы по компонентам – это превращает изменения API в проход по сотне файлов.

КАК РАБОТАЕТ

- 01 `shared/api/client` – базовый `fetch`-клиент (`baseUrl`, `auth`, `retry`, `timeout`, ошибки).
- 02 `entities/user/api/userApi.ts` – методы `getUser`, `updateUser`.
- 03 `features/auth/api/loginApi.ts` – методы `login`, `logout`, `refresh`.
- 04 Возвращают ТИПИЗИРОВАННЫЕ результаты, а не `raw Response`.
- 05 Контракт `DTO ↔ domain` – через адаптер на этом же слое.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой `fetch` к серверу.
- 02 GraphQL-операции (через `codegen`).
- 03 `WebSocket`-подписки.

ПОДВОДНЫЕ КАМНИ

- 01 `fetch` внутри компонента – невозможно протестировать.
- 02 API-вызов через много слоёв сразу – потеря контроля.
- 03 Изменение `endpoint` – правка в десятках файлов.
- 04 Тип `unknown` без валидации – `runtime`-баги.

КОД

// Скелет

```
// shared/api/client.ts
export const apiClient = createClient({ baseUrl: '/api' });

// entities/user/api/userApi.ts
export const userApi = {
  getById: (id: string) => apiClient.get<User>(`/users/${id}`),
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Что должен делать базовый клиент?
- 02 Где жить адаптерам `DTO ↔ domain`?
- 03 Чем GraphQL-клиент отличается?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs/reference/layers>

ДЕНЬ 2

НАГРУЗКА

СРЕДНИЙ

4 ВОПРОСА

ГДЕ ЖИТЬ**HOOKS****DOMAIN****UI****DESIGN SYSTEM****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Куда складывать хуки, сервисы, компоненты, и зачем DS.

ВОПРОСЫ ДНЯ

Q06 Где должны жить hooks?

Q07 Где должны жить domain services?

Q08 Где должны жить UI components?

Q09 Как проектировать design system?

Где должны жить hooks?

ОПИСАНИЕ

Хуки распределяются по слоям так же, как и UI: общие (`useDebounce`, `useLocalStorage`) – в `shared/hooks`; доменные (`useUser`, `useUserPosts`) – в `entities`; логика конкретной фичи (`useLoginForm`) – в `features`. Кастомные хуки – самый удобный способ инкапсулировать и переиспользовать `stateful`-логику между компонентами.

КАК РАБОТАЕТ

- 01 `shared/hooks` – `useDebounce`, `useThrottle`, `useMediaQuery`, `useLocalStorage`.
- 02 `entities/user/model/useUser` – обёртка вокруг `userApi` + `react-query`.
- 03 `features/auth/model/useLogin` – handle login form, submit, validation.
- 04 `pages/...page.tsx` – комбинирует хуки `features`.
- 05 Принцип: один хук – одна ответственность.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Везде, где есть переиспользуемая логика.
- 02 Когда внутри компонента >50 строк `stateful`-логики – пора в хук.
- 03 Когда нужно покрыть тестами без UI.

ПОДВОДНЫЕ КАМНИ

- 01 Хук, который делает 5 разных вещей – нечитаем.
- 02 Хук, зависящий от много контекстов – сложно тестировать.
- 03 Хук в компоненте без префикса `use` – ESLint правило `react-hooks` не работает.
- 04 Side effects на `mount` без `cleanup` – утечки.

КОД

// Хук фичи

```
// features/auth/model/useLogin.ts
export function useLogin() {
  const [error, setError] = useState<string | null>(null);
  const m = useMutation({
    mutationFn: authApi.login,
    onError: (e) => setError(e.message),
  });
  return { login: m.mutate, error, isPending: m.isPending };
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда хук слишком тяжёлый и пора рефакторить?

- 02 Чем тестирование хука отличается от компонента?
- 03 Где жить хуку, который нужен в разных features?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/hooks>

Где должны жить domain services?

ОПИСАНИЕ

Domain services – функции/классы, реализующие бизнес-логику (расчёт цены, валидация заказа, правила конвертации валют). Живут на уровне entity или features, не в UI и не в shared. Изолированы – без зависимостей от React, легко тестируются юнит-тестами.

КАК РАБОТАЕТ

- 01 entities/order/lib/calculateTotal.ts – pure-функция.
- 02 entities/order/model/orderService.ts – класс с методами.
- 03 Не импортируют из react-query / redux / ui.
- 04 Принимают данные, возвращают результат – без сайд-эффектов или с явными.
- 05 Используются хуками и компонентами как pure utilities.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая бизнес-логика, которую нужно тестировать отдельно от UI.
- 02 Сложные расчёты, валидации, преобразования.
- 03 Стратегии (Strategy pattern) – выбор алгоритма по типу.

ПОДВОДНЫЕ КАМНИ

- 01 Доменная логика, размазанная по компонентам – нельзя переиспользовать.
- 02 Сервис, импортирующий react – теряет тестируемость.
- 03 Слишком объёмный сервис «всё про заказы» – пора разбивать.
- 04 Связь с конкретной БД/API внутри сервиса – нарушает изоляцию.

КОД

// Pure-функция

```
// entities/order/lib/calculateTotal.ts
export function calculateTotal(items: Item[], coupon?: Coupon): number {
  const sub = items.reduce((a, x) => a + x.price * x.qty, 0);
  return coupon ? sub * (1 - coupon.discount) : sub;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем pure-функция отличается от хука?
- 02 Когда сервис превращается в класс?
- 03 Где тестировать бизнес-логику?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs/reference/layers>

ВОПРОС Q08

Где должны жить UI components?

ОПИСАНИЕ

UI распределяется по уровням: `shared/ui` – примитивы (`Button`, `Input`, `Modal`, `Spinner`); `entities/*/ui` – представление сущности (`UserCard`); `features/*/ui` – интерактивные элементы (`LoginForm`); `widgets` – композиции (`Header`); `pages` – финальная сборка. Каждый уровень переиспользуется тем, что выше.

КАК РАБОТАЕТ

- 01 `shared/ui` – без бизнес-знания, только props в стиле design system.
- 02 `entities/<name>/ui` – карточки/строки сущности с минимумом поведения.
- 03 `features/<name>/ui` – формы и кнопки с действиями.
- 04 `widgets` – `Header`, `Sidebar`, `FilterPanel`.
- 05 `pages/<name>/ui` – финальная страница с layout.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Каждый новый компонент – чёткий ответ, в каком слое.
- 02 При выносе из `inline` в отдельный файл.
- 03 При переиспользовании нескольких раз.

ПОДВОДНЫЕ КАМНИ

- 01 Кнопка с `props.icon=BellIcon` в `shared` – это уже не `shared`.
- 02 Карточка пользователя в `shared/ui` – нет, это `entity`.
- 03 `Inline`-стили в `shared/ui` без поддержки темы.
- 04 Не предусмотренный API: `shared-Button` с `props.iconLeft`, `iconRight`, ребёнок – слишком много.

КОД

// Различие

```
// shared/ui/Button.tsx – без знания о домене
<Button variant="primary">Save</Button>

// entities/user/ui/UserCard.tsx – про юзера
<UserCard user={u} />

// features/auth/ui/LoginForm.tsx – про действие
<LoginForm onSuccess={...} />
```

МОГУТ ДОСПРОСИТЬ

- 01 Где жить иконкам?

02 Чем widget отличается от feature?

03 Куда положить Layout-компонент?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs/reference/layers>

ВОПРОС Q09

Как проектировать design system?

ОПИСАНИЕ

Design System – набор согласованных UI-примитивов, токенов (цвета, размеры, типографика) и паттернов. Принципы: единый источник правды (Figma + код), минимально-достаточный API, версионирование, документация (Storybook). Чаще всего реализуется как отдельный пакет, который потребляют приложения.

КАК РАБОТАЕТ

- 01 Tokens: цвета, отступы, шрифты, радиусы – переменные (CSS vars / TypeScript constants).
- 02 Components: Button, Input, Modal, Tabs, Tooltip, DataTable.
- 03 Patterns: формы, диалоги, таблицы – composed-компоненты.
- 04 Storybook – каталог + interactive playground.
- 05 Версионирование semver, change-log.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Множество приложений у одной команды.
- 02 Большой продукт с длинной историей.
- 03 Когда дизайн меняется – нужно одно место для правок.
- 04 B2B / SaaS с несколькими white-label.

ПОДВОДНЫЕ КАМНИ

- 01 DS, который не используется реальными UI – мёртвый.
- 02 Слишком кастомизированный API – превращается в complexity-bomby.
- 03 Без Figma-токенов – рассинхрон с дизайнерами.
- 04 Без версионирования – обновления ломают потребителей.

КОД

// Token-переменные

```
/* tokens.css */
:root {
  --color-primary: #1A73E8;
  --space-2: 8px;
  --radius-md: 6px;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем headless-UI помогает в DS?
- 02 Как версионировать DS?

03 Что хранить в Figma vs в коде?

ПОЛНАЯ СТАТЬЯ

<https://www.designsystems.com/>

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

3 ВОПРОСА

DOMAIN ↔ UI**DATA FETCHING ↔ RENDER****TABLE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Главный навык — отделять предметную логику от рендера.

ВОПРОСЫ ДНЯ

Q10 Как отделять domain logic от UI?

Q11 Как отделять data fetching от rendering?

Q12 Как проектировать reusable table?

Как отделять domain logic от UI?

ОПИСАНИЕ

Принцип: компоненты только рендерят и обрабатывают ввод; решения, расчёты и бизнес-правила – в domain-services / hooks. Компонент должен быть «глупым» – получил props, отдал JSX. Это даёт тестируемость (unit-тесты для логики, separately от UI), переиспользуемость (логика работает в Web и RN).

КАК РАБОТАЕТ

- 01 Container vs Presentational pattern (старый, но рабочий).
- 02 Hook извлекает state-логику, компонент только рендерит.
- 03 Pure-функции для расчётов.
- 04 useReducer для сложных переходов state.
- 05 Adapter-pattern: data-layer трансформирует raw в domain-форму.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой нетривиальный компонент с логикой.
- 02 Code review – сигнал «логика в JSX» = пора рефакторить.
- 03 Тестирование – UI без логики тестировать проще.
- 04 Переиспользование между Web и Native.

ПОДВОДНЫЕ КАМНИ

- 01 Логика прямо в onClick / useEffect – нечитаемо.
- 02 Хук-комбайн «всё про страницу» – невозможно переиспользовать.
- 03 Бизнес-логика в shared/ui – нарушение слоёв.
- 04 Чрезмерное разделение → много мелких файлов.

КОД

// До и после

```
// до – логика в компоненте
function CartPage() {
  const total = items.reduce((a, x) => a + x.price * x.qty, 0);
  const tax = total * 0.2;
  return <div>Итого: {total + tax}</div>;
}

// после – pure-функция
function CartPage() {
  const total = calculateTotal(items);
  return <div>Итого: {total}</div>;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое presentational vs container?
- 02 Куда поместить calculateTotal?
- 03 Когда логику оставить в компоненте?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs/reference/layers>

Как отделять data fetching от rendering?

ОПИСАНИЕ

Используй data-fetching библиотеку (react-query / SWR / Apollo) и инкапсулируй вызовы в хуки на уровне entities/features. Компонент не делает fetch, он использует хук, получает {data, isLoading, error} и рендерит. Это позволяет легко менять источник данных, тестировать UI с моками.

КАК РАБОТАЕТ

- 01 API-клиент в entities/<name>/api.
- 02 Хук в entities/<name>/model/useEntityName.
- 03 Компонент использует хук, рендерит data.
- 04 Suspense-режим – данные гарантированы внутри границы.
- 05 Error-boundary рядом – обрабатывает promise-reject.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая компонентная страница с fetch.
- 02 Тесты – UI с моком данных через MSW.
- 03 Замена REST на GraphQL – один хук, не каждый компонент.

ПОДВОДНЫЕ КАМНИ

- 01 fetch внутри useEffect → race condition (см. соседнюю тему).
- 02 Данные в Redux + react-query одновременно → дубли.
- 03 Без abstraction слой UI завязан на конкретный URL.
- 04 Без Suspense / loading-состояний UI «дёргается».

КОД

// Хук + компонент

```
// entities/user/model/useUser.ts
export function useUser(id: string) {
  return useQuery({ queryKey: ['user', id], queryFn: () => userApi.getById(id) });
}

// pages/UserPage.tsx
function UserPage({ id }: { id: string }) {
  const { data, isLoading } = useUser(id);
  if (isLoading) return <Spinner />;
  return <UserCard user={data} />;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-query упрощает архитектуру?
- 02 Куда положить трансформацию ответа?
- 03 Как мокать данные в тестах?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest>

Как проектировать reusable table?

ОПИСАНИЕ

Generic-компонент, принимающий `rows: T[]` и `columns: Column<T>[]`. Поведение (сортировка, пагинация, фильтры, выделение) – через props или controlled API. Готовые библиотеки (TanStack Table) решают 80% задач без переизобретения. Для production-grade нужны: virtualization, sticky header, accessibility.

КАК РАБОТАЕТ

- 01 Generic <T> для типобезопасных колонок.
- 02 Column<T> = { key/accessor, header, render?, sortable? }.
- 03 Sorting/filtering – controlled через onSortChange / onFilterChange.
- 04 Pagination/infinite scroll – ещё один уровень API.
- 05 Виртуализация для длинных таблиц (react-virtual / TanStack Virtual).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дашборды.
- 02 Админ-панели.
- 03 Списки заказов / пользователей.
- 04 Любые табличные данные.

ПОДВОДНЫЕ КАМНИ

- 01 Большие таблицы без виртуализации – лагают.
- 02 Inline-render-функции пересоздаются – мемоизация ломается.
- 03 Accessibility (aria-roles, table semantics) часто забывают.
- 04 Слишком гибкий API – никто не понимает.

КОД

// Скелет API

```
type Column<T> = { key: keyof T; header: string; render?: (r: T) => ReactNode };
function Table<T>({ rows, columns }: { rows: T[]; columns: Column<T>[] }) {
  return (
    <table>{/* ... */}</table>
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем TanStack Table выгоден?
- 02 Как добавить виртуализацию?
- 03 Какие aria-роли в таблице?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/table/latest>

FORM**WIZARD****DASHBOARD****LOADING/EMPTY/ERROR****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Типовые UI-паттерны и как не превратить их в ад.

ВОПРОСЫ ДНЯ

Q13 Как проектировать complex form?

Q14 Как проектировать wizard flow?

Q15 Как проектировать dashboard?

Q21 Как проектировать loading / empty / error states?

ВОПРОС Q13

Как проектировать complex form?

ОПИСАНИЕ

Сложная форма – это валидация, conditional fields, multi-step, async-проверки, optimistic-сохранение. Тулинг: react-hook-form + Zod (для типобезопасности и валидации). Архитектурно: схема валидации отдельно, контейнер-формы отдельно, поля как мелкие компоненты.

КАК РАБОТАЕТ

- 01 react-hook-form: register, handleSubmit, formState.
- 02 Zod: schema-first → tip из z.infer.
- 03 zodResolver – связка.
- 04 Conditional fields через watch().
- 05 Field-Components: <TextField name="x" /> – обёртки над input.
- 06 Submit handler возвращает Promise – onSubmit ждёт.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Регистрация / профиль / настройки.
- 02 Формы заказа, оплаты.
- 03 Конструкторы.
- 04 Многошаговые wizard'ы.

ПОДВОДНЫЕ КАМНИ

- 01 Без runtime-валидации – баги "strings вместо numbers".
- 02 Все поля controlled – ре-рендер на каждое нажатие.
- 03 Слишком много conditional – сложно поддерживать.
- 04 Валидация на submit, а не на blur – UX страдает.

КОД

// react-hook-form + Zod

```
const Schema = z.object({
  email: z.string().email(),
  age: z.number().min(18),
});
type FormValues = z.infer<typeof Schema>;
const { register, handleSubmit, formState: { errors } } =
  useForm<FormValues>({ resolver: zodResolver(Schema) });
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-hook-form быстрее controlled подхода?

02 Где валидировать – клиент vs сервер?

03 Как делать conditional fields?

ПОЛНАЯ СТАТЬЯ

<https://react-hook-form.com/>

ВОПРОС Q14

Как проектировать wizard flow?

ОПИСАНИЕ

Wizard – multi-step форма с явными шагами, навигацией (вперёд/назад), валидацией каждого шага, состоянием между шагами. Архитектура – state-machine (DU состояний или XState), отдельный контейнер-логики, step-components. Можно сохранять progress в URL для shareable-ссылки.

КАК РАБОТАЕТ

- 01 Шаги как DU: { step: 'address' } | { step: 'payment' } | { step: 'review' }.
- 02 Контейнер хранит общий state и логику переходов.
- 03 Каждый шаг – компонент, принимающий state + onNext + onBack.
- 04 Валидация каждого шага через Zod.
- 05 URL: ?step=payment – для возврата в нужный шаг.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Onboarding пользователя.
- 02 Checkout-флоу.
- 03 Многошаговая регистрация.
- 04 Мастера настройки.

ПОДВОДНЫЕ КАМНИ

- 01 Один большой объект state без структуры – путаница.
- 02 Шаги жёстко зашиты – нельзя переставить.
- 03 Backend-side флоу не синхронизирован с frontend.
- 04 Refresh теряет state – нужен localStorage / URL.

КОД

// DU состояний

```
type Wizard =  
  | { step: 'info';    data: Partial<UserInfo> }  
  | { step: 'address'; data: UserInfo & Partial<Address> }  
  | { step: 'review';  data: FullData };
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем XState помогает в wizard?
- 02 Как сохранять draft между сеансами?
- 03 Что делать с back-button?

ПОЛНАЯ СТАТЬЯ

<https://state.ly/ai/docs>

ВОПРОС Q15

Как проектировать dashboard?

ОПИСАНИЕ

Дашборд — набор виджетов с независимыми источниками данных. Принципы: каждый виджет — отдельная feature-граница; данные через react-query (с разными staleTime); layout — grid-system; partial loading (виджет может загрузиться отдельно от других); error-boundary вокруг каждого виджета — чтобы один не обвалил всю страницу.

КАК РАБОТАЕТ

- 01 Виджеты — независимые компоненты features или widgets.
- 02 Каждый сам грузит свои данные через хук + react-query.
- 03 Layout — CSS Grid / Flex; responsive.
- 04 ErrorBoundary вокруг каждого виджета.
- 05 Suspense / fallback — для прогрессивной загрузки.
- 06 Конфигурация дашборда (какие виджеты, где) — в store/URL.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Аналитические панели.
- 02 Админ-инструменты.
- 03 Личные кабинеты.

ПОДВОДНЫЕ КАМНИ

- 01 Один общий fetch для всех виджетов — медленнее, хрупче.
- 02 Без error-boundary — один сбой роняет всё.
- 03 Тяжёлые charts без виртуализации.
- 04 Дублирование данных между виджетами.

КОД

// Скелет

```
<Grid>
  <ErrorBoundary><Suspense fallback={<Spinner />}>
    <RevenueWidget />
  </Suspense></ErrorBoundary>
  <ErrorBoundary><Suspense fallback={<Spinner />}>
    <UsersWidget />
  </Suspense></ErrorBoundary>
</Grid>
```

МОГУТ ДОСПРОСИТЬ

- 01 Где хранить layout пользователя?

- 02 Как избежать дублирования данных?
- 03 Чем widget отличается от чарта?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/docs/reference/layers>

ВОПРОС Q 21

Как проектировать loading / empty / error states?

ОПИСАНИЕ

Все асинхронные UI должны явно описать четыре состояния: `idle` (ничего не запросили), `loading` (грузим), `success with data / empty` (пусто), `error`. Без `empty-state` UI выглядит сломанным; без `error-state` – пугает. Лучшая практика – declarative-обёртка: `<AsyncBlock state={...} renderEmpty renderError renderData />`.

КАК РАБОТАЕТ

- 01 Discriminated union state.
- 02 Skeleton-loaders вместо crude Spinner.
- 03 Empty-state с СТА («Добавьте первый элемент»).
- 04 Error-state с retry-кнопкой и понятным сообщением.
- 05 Suspense + ErrorBoundary как декларативный механизм.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой fetch UI.
- 02 Списки, дашборды, профили, поиск.
- 03 Формы загрузки файлов.

ПОДВОДНЫЕ КАМНИ

- 01 Empty-state == пустой блок без объяснения = bug-репорт.
- 02 Error без retry = пользователь застрял.
- 03 Spinner на всю страницу при подгрузке маленького блока.
- 04 Skeleton, не похожий на финальный UI.

КОД

// Декларативный AsyncBlock

```
function AsyncBlock<T>({ q, renderEmpty, children }: { q: UseQueryResult<T[]>; renderEmpty: ReactNode; children: (d: T[]) => ReactNode }) {
  if (q.isLoading) return <SkeletonList />;
  if (q.error)      return <Error onRetry={q.refetch} />;
  if (!q.data?.length) return <>{renderEmpty}</>;
  return <>{children(q.data)}</>;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем skeleton лучше spinner?
- 02 Что писать в empty-state?
- 03 Как retry с backoff?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Suspense>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 5 ВОПРОСОВ

ROLES**PERMISSIONS****UNAUTHORIZED****ERROR BOUNDARIES**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Безопасность UI и устойчивость к ошибкам.

ВОПРОСЫ ДНЯ

- Q16 Как проектировать role-based UI?
- Q17 Как проектировать frontend permissions?
- Q18 Как защититься от unauthorized UI states?
- Q19 Как организовать error boundaries?
- Q20 Где ставить error boundaries?

Как проектировать role-based UI?

ОПИСАНИЕ

Роли пользователя определяют, что он видит и что может. Архитектура – auth-store с current user + roles + permissions, хуки useCan(action), декларативные обёртки <Can action="...">. Сервер обязательно перепроверяет – клиент только UX. Защищённые маршруты и условный рендер по роли.

КАК РАБОТАЕТ

- 01 auth-store: { user, roles: Set<Role>, permissions: Set<Permission> }.
- 02 useRole(), useCan(action) – хуки.
- 03 <RouteGuard role="admin"> – обёртка маршрута.
- 04 Server-side check – обязательное дублирование.
- 05 Permissions грузятся при login, инвалидируются при logout/role-change.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Admin-панели.
- 02 Multi-tenant SaaS.
- 03 B2B-приложения.
- 04 Любые системы с ACL.

ПОДВОДНЫЕ КАМНИ

- 01 Только клиентская проверка – security hole.
- 02 Жёсткое if (role === 'admin') разбросано по 100 файлам.
- 03 Изменение прав не отражается в UI без relogin.
- 04 Невидимые элементы продолжают работать через keyboard shortcut.

КОД

// Декларативный Can

```
function Can({ action, children, fallback = null }: { action: string; children: ReactNode; fallback?: ReactNode }) {
  const can = useCan(action);
  return can ? <{children}> : <{fallback}>;
}
// <Can action="user.delete"><DeleteBtn /></Can>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем Can лучше inline-if?
- 02 Как обновлять permissions без relogin?
- 03 Когда ABAC лучше RBAC?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/conditional-rendering>

Как проектировать frontend permissions?

ОПИСАНИЕ

Permission model – структура, описывающая «кто что может». RBAC: роли → permissions; ABAC: атрибуты пользователя + объекта → правила. Хранится в auth-store, проверяется через хуки/обёртки. Frontend ОТРАЖАЕТ серверные правила – реальная безопасность только на бэкенде.

КАК РАБОТАЕТ

- 01 RBAC: роли + матрица прав.
- 02 ABAC: правила вида (user.dept === resource.dept) && (user.role === 'manager').
- 03 Permission-checker как pure-функция.
- 04 Хук useCan, обёртка <Can>.
- 05 Cache permissions с invalidation.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Enterprise B2B.
- 02 Multi-tenant SaaS.
- 03 Системы с владением ресурсами (owner-based).
- 04 Финтех/HR – много ролей.

ПОДВОДНЫЕ КАМНИ

- 01 Frontend-only check ≠ безопасность.
- 02 Глубокая иерархия ролей сложна для UX.
- 03 Переход между ролями не reflect-ится сразу.
- 04 Большие матрицы прав становятся неподдерживаемыми вручную – нужен provider (Casbin, Auth0).

КОД

```
// Хук useCan

export function useCan(action: string): boolean {
  const perms = useAuthStore((s) => s.permissions);
  return perms.has(action);
}
```

МОГУТ ДОСПРОСИТЬ

- 01 RBAC vs ABAC – что выбрать?
 - 02 Где хранить permissions?
 - 03 Как тестировать permission-логику?
-

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#literal-types>

Как защититься от unauthorized UI states?

ОПИСАНИЕ

Меры: route guards (защищённые маршруты редиректят неавторизованных), декларативные `<Can>`-обёртки вокруг чувствительных элементов, серверная проверка на каждом действии (не доверять frontend), валидация permissions при mount страницы, error-boundary вокруг защищённых блоков.

КАК РАБОТАЕТ

- 01 RouteGuard: redirect → /login если no auth.
- 02 `<Can>` обёртка перед UI-элементами.
- 03 Сервер всегда проверяет – даже если фронт не послал запрос.
- 04 При смене role/logout – invalidate всё что зависит.
- 05 Логирование попыток несанкционированного доступа.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Все защищённые маршруты.
- 02 Все чувствительные действия (delete, transfer).
- 03 Все экраны с PII.

ПОДВОДНЫЕ КАМНИ

- 01 Скрыть UI ≠ защитить – кнопка скрыта, но XHR работает.
- 02 Кэш в браузере хранит данные после logout.
- 03 JWT в localStorage уязвим к XSS.
- 04 Permissions грузятся асинхронно – flash of unauthorized content.

КОД

// RouteGuard

```
function ProtectedRoute({ children }: { children: ReactNode }) {  
  const user = useAuthStore((s) => s.user);  
  if (!user) return <Navigate to="/login" />;  
  return <>{children}</>;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flash of unauthorized content?
- 02 Как очищать кэш при logout?
- 03 Где хранить токен безопасно?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-project-top-ten/>

Как организовать error boundaries?

ОПИСАНИЕ

Error boundary – React-компонент, который ловит ошибки render/lifecycle/constructor child-tree и показывает fallback. Обязателен на верхних уровнях для предотвращения white screen, и желателен вокруг крупных секций для partial-degradation. Не ловит: события, async-код, errors в самом boundary.

КАК РАБОТАЕТ

- 01 Class component с componentDidCatch / getDerivedStateFromError.
- 02 react-error-boundary – современная functional обёртка.
- 03 Fallback-UI: friendly message + retry + report.
- 04 onError callback – отправка в Sentry.
- 05 Несколько уровней boundary: app-level, section-level.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 App-level – на самом верху, гарантия не-упасть.
- 02 Section-level – вокруг каждого виджета дашборда.
- 03 Lazy-loaded routes – на случай chunk-load-error.
- 04 Suspense + ErrorBoundary всегда вместе.

ПОДВОДНЫЕ КАМНИ

- 01 Не ловит async/event errors – нужны другие обработчики.
- 02 Слишком крупный boundary → одна ошибка убивает большой блок.
- 03 Без onError → ошибки не попадают в monitoring.
- 04 Fallback с навигацией внутри сам может сломаться.

КОД

```
// react-error-boundary
```

```
import { ErrorBoundary } from 'react-error-boundary';  
  
<ErrorBoundary  
  FallbackComponent={({ error, resetErrorBoundary }) => (  
    <div>  
      <p>Что-то пошло не так</p>  
      <button onClick={resetErrorBoundary}>Повторить</button>  
    </div>  
  )}  
  onError={(e) => Sentry.captureException(e)}  
>  
  <App />  
</ErrorBoundary>
```

МОГУТ ДОСПРОСИТЬ

- 01 Что boundary НЕ ловит?
- 02 Как обрабатывать async-ошибки?
- 03 Чем react-error-boundary удобнее class-component?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>

Где ставить error boundaries?

ОПИСАНИЕ

Минимум — на верхнем уровне приложения (app-level). Дополнительно — вокруг каждого крупного независимого блока (виджет, секция, lazy-роут), чтобы одна ошибка не «сбивала» всё. Гранулярность зависит от UX: каждый виджет дашборда — своя boundary; каждая колонка чата — своя.

КАК РАБОТАЕТ

- 01 App-level — гарантия от white screen.
- 02 Route-level — для lazy-загруженных страниц.
- 03 Section-level — вокруг виджетов / больших фич.
- 04 Suspense рядом с boundary.
- 05 Onerror callback в каждом boundary — для логирования.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая SPA.
- 02 Дашборды.
- 03 Микро-frontend-композиции.
- 04 Lazy-loaded маршруты.

ПОДВОДНЫЕ КАМНИ

- 01 Один app-boundary — слишком грубо.
- 02 Boundary на каждом компоненте — overkill.
- 03 Fallback-UI без CTA → пользователь застрял.
- 04 Boundary в самой себе ломается → infinite loop.

КОД

// Многоуровневая стратегия

```
<App>
  <ErrorBoundary>          {/* app-level */}
    <Routes>
      <Route path="/dashboard" element={
        <ErrorBoundary>    {/* page-level */}
          <Dashboard />
        </ErrorBoundary>
      } />
    </Routes>
  </ErrorBoundary>
</App>
```

МОГУТ ДОСПРОСИТЬ

- 01 Сколько уровней boundary оптимально?
- 02 Как boundary взаимодействует с Suspense?
- 03 Что показывать в fallback?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>

DEGRADATION**FLAGS****ROLLOUT****i18n****THEMING****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Релизы по чуть, языки и темы.

ВОПРОСЫ ДНЯ

- Q22 Как проектировать graceful degradation?
- Q23 Как проектировать feature flags?
- Q24 Как делать gradual rollout на frontend?
- Q25 Как организовать i18n?
- Q26 Как проектировать theming?

ВОПРОС Q22

Как проектировать graceful degradation?

ОПИСАНИЕ

Graceful degradation – приложение работает (с урезанным функционалом) даже когда часть зависимостей недоступна. Стратегии: feature flags (часть фич отключена), fallback-данные (cached / static), partial UI (виджет упал – остальные показываются), offline-mode, skeleton + retry.

КАК РАБОТАЕТ

- 01 ErrorBoundary вокруг каждого крупного блока.
- 02 Cached/stale-данные при ошибке fresh-fetch.
- 03 Fallback-UI для отсутствующих данных.
- 04 Service Worker – offline cache.
- 05 Health check API – отключение зависящих фич.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production-приложения с критичными UX.
- 02 PWA / mobile-friendly сайты.
- 03 Realtime-приложения с нестабильной сетью.
- 04 Микро-frontend, где сервисы могут быть недоступны.

ПОДВОДНЫЕ КАМНИ

- 01 Stale-данные показываются без указания «старые».
- 02 Offline-mode без sync → конфликты.
- 03 Service Worker слишком жёстко кэширует – пользователь видит старую версию.
- 04 Без telemetry не понятно, как часто работает degradation.

КОД

```
// Cached fallback
```

```
useQuery({
  queryKey: ['x'],
  queryFn: api.x,
  placeholderData: (prev) => prev,    // показать предыдущие при refetch
  retry: 3,
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое stale-while-revalidate?
- 02 Зачем Service Worker?
- 03 Как уведомить пользователя о degradation?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/offline-cookbook>

В О П Р О С Q 2 3

Как проектировать feature flags?

ОПИСАНИЕ

Feature flags – конфиг включения/выключения фичи без релиза. Источник: provider (LaunchDarkly, GrowthBook, Unleash) или custom. Хранятся в auth-store / FlagsProvider. Используются через хук useFlag(key), обёртку <Feature flag="..."> или conditional render. Все ключи – typed union для отлова опечаток.

КАК РАБОТАЕТ

- 01 Загрузка флагов на старте (или server-side).
- 02 Кэш + invalidation при login/logout.
- 03 useFlag(key: FlagKey): boolean.
- 04 <Feature flag="newCheckout">...</Feature>.
- 05 Default-значение при недоступности provider.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 A/B-тесты.
- 02 Gradual rollout.
- 03 Kill switch.
- 04 Dark launches.

ПОДВОДНЫЕ КАМНИ

- 01 Mutual flag dependencies → сложная матрица.
- 02 Forgotten flags → код-долг.
- 03 Слишком гранулярные флаги → fragile UI.
- 04 Без тестирования с разными комбинациями – баги.

КОД

// Typed flags

```
type FlagKey = 'newCheckout' | 'darkMode2';
function Feature({ flag, children }: { flag: FlagKey; children: ReactNode }) {
  return useFlag(flag) ? <>{children}</> : null;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем GrowthBook лучше env vars?
- 02 Как сделать audit "что включено у user"?
- 03 Когда удалять флаг?

ПОЛНАЯ СТАТЬЯ

<https://docs.growthbook.io/>

ВОПРОС Q24

Как делать gradual rollout на frontend?

ОПИСАНИЕ

Gradual rollout – выкатка новой фичи на постепенно растущий процент пользователей. Реализация: feature flag с правилом «10% пользователей», постепенное увеличение до 100%. Параллельно метрики (errors, engagement) сравниваются между группами. Если метрика хуже – откат. Это и есть «canary deploy» на уровне клиентского кода.

КАК РАБОТАЕТ

- 01 Provider типа LaunchDarkly/GrowthBook поддерживает rollout %.
- 02 Targeting rules: по user-id, plan, country.
- 03 Metric tracking – A/B-сравнение.
- 04 Канареечный релиз: 1% → 10% → 50% → 100%.
- 05 Kill switch для мгновенного отката.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие фичи с риском.
- 02 Изменения checkout, payments – критичные пути.
- 03 Performance-эксперименты.
- 04 UI-редизайны.

ПОДВОДНЫЕ КАМНИ

- 01 Без метрик rollout вслепую → можно не заметить деградацию.
- 02 Targeting по cookie → банальные refresh ломают группу.
- 03 Сегмент 1% слишком мал для статзначимости.
- 04 Rollout не закрытый = «навсегда 50%» = долг.

КОД

```
// Targeting в provider
// в GrowthBook conditions:
// rollout: 10% of users
// targeting: country === 'RU' && plan === 'pro'
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое canary deploy?
- 02 Какие метрики сравнивать?
- 03 Как сделать rollback быстро?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/CanaryRelease.html>

ВОПРОС Q25

Как организовать i18n?

ОПИСАНИЕ

Интернационализация – поддержка нескольких языков, локалей дат/чисел, RTL. Стандарт – react-i18next / FormatJS. Принципы: ключи переводов, словари по локалям, lazy-load локалей, ICU MessageFormat для плюрализации, locale-detection (URL / browser / user preference).

КАК РАБОТАЕТ

- 01 Ключ → перевод: `t('login.title')`.
- 02 Словари в JSON, лежат в `public/locales` или встраиваются.
- 03 Lazy-load: догрузка только нужного языка.
- 04 ICU MessageFormat: плюрализация, gender, numbers.
- 05 Date/Number форматы – Intl API.
- 06 RTL: `dir="rtl"` + CSS logical properties.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой международный продукт.
- 02 B2C с пользователями из нескольких регионов.
- 03 B2B с европейскими/азиатскими клиентами.

ПОДВОДНЫЕ КАМНИ

- 01 Хардкод текста по всему UI – рефакторинг сложен.
- 02 Плюрализация в шаблонах typeof «1 яйцо / 2 яйца» – ICU.
- 03 RTL ломает CSS, если используются left/right вместо start/end.
- 04 Date.format с фиксированной локалью.

КОД

```
// react-i18next
const { t } = useTranslation();
<h1>{t('login.title')}</h1>
<p>{t('cart.items', { count: items.length })}</p>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем ICU MessageFormat отличается от sprintf?
- 02 Как поддерживать RTL?
- 03 Где хранить словари?

ПОЛНАЯ СТАТЬЯ

<https://www.i18next.com/>

ВОПРОС Q26

Как проектировать theming?

ОПИСАНИЕ

Темизация – переключение визуального стиля без изменения логики. Реализация: токены (CSS variables), ThemeProvider, переключатель + sync с system-preference + сохранение в storage. SSR-важно: тема загружается до первой отрисовки, иначе flash of wrong theme. Поддержка минимум: light/dark, опционально – несколько кастомных.

КАК РАБОТАЕТ

- 01 CSS variables как tokens (--color-bg).
- 02 data-theme="dark" атрибут на html.
- 03 matchMedia('(prefers-color-scheme: dark)') – детект system.
- 04 localStorage – сохранение выбора.
- 05 Inline-script в head – задаёт тему до hydration.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сайты с light/dark.
- 02 White-label продукты с brand-colors клиентов.
- 03 Accessibility (high contrast).
- 04 Эксперименты с дизайном.

ПОДВОДНЫЕ КАМНИ

- 01 Flash of wrong theme при SSR – критичный UX-баг.
- 02 Hardcoded colors в компонентах – не поддаются темизации.
- 03 Слишком много темы – сложно тестировать.
- 04 tailwind с jit – нужно явно перечислить варианты.

КОД

// Inline-script для SSR

```
<script>
  (function() {
    var t = localStorage.getItem('theme') || 'light';
    document.documentElement.setAttribute('data-theme', t);
  })();
</script>
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flash of wrong theme?
- 02 Как сделать theming в Next App Router?
- 03 Чем CSS-переменные удобнее JS-токенов?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/color-scheme>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

9 ВОПРОСОВ

RESPONSIVE**SHARED API****DRILLING****REVIEW****ADR****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Качество кода и процесс принятия решений.

ВОПРОСЫ ДНЯ

- Q27 Как проектировать responsive layout?
- Q28 Как проектировать shared component API?
- Q29 Как избежать prop drilling?
- Q30 Как избежать over-abstraction?
- Q31 Как проводить frontend architecture review?
- Q32 Как документировать frontend architecture?
- Q33 Как принять архитектурное решение?
- Q34 Когда писать ADR?
- Q35 Как оценивать архитектурный trade-off?

Как проектировать responsive layout?

ОПИСАНИЕ

Mobile-first: базовый стиль для маленького экрана, media-queries добавляют десктопные улучшения. Tailwind / CSS – sm:/md:/lg: модификаторы. Контейнер-queries для component-level adaptivity (новые браузеры). Важно: тач-цели $\geq 44\text{px}$, читаемый шрифт, отсутствие горизонтального скrolла.

КАК РАБОТАЕТ

- 01 Базовый стиль – для мобилки.
- 02 @media (min-width) – добавления для больших экранов.
- 03 @container – реакция на размер контейнера, не viewport.
- 04 Flex / Grid – основные инструменты.
- 05 Adaptive typography (clamp, vw-units).
- 06 Touch-friendly: min 44px.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой web-проект с пользователями на разных устройствах.
- 02 PWA / mobile-friendly сайты.
- 03 Адаптивные дашборды.

ПОДВОДНЫЕ КАМНИ

- 01 Desktop-first → ад на мобильнике.
- 02 Hover-only интерактив на тач-устройствах.
- 03 Hardcoded px без clamp/rem – плохая accessibility.
- 04 Слишком много breakpoints – сложно поддерживать.

КОД

// Mobile-first Tailwind

```
<div className="p-4 text-base sm:p-6 sm:text-lg lg:p-8">
  <Sidebar className="hidden md:block" />
  <Main />
</div>
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое container queries?
 - 02 Как работать с touch vs hover?
 - 03 Какие минимальные размеры тач-целей?
-

ПОЛНАЯ СТАТЬЯ

<https://web.dev/learn/design>

Как проектировать shared component API?

ОПИСАНИЕ

Принципы: минимальный API (только то, что реально нужно), compound-components для гибкости (Tabs.Tab, Tabs.Panel), render-props или children-as-function для кастомизации, sensible defaults. Не пытаться угадать все use-cases – оставлять «escape hatches» (className, as).

КАК РАБОТАЕТ

- 01 Minimal-default props.
- 02 Compound: `<Modal><Modal.Header /><Modal.Body /></Modal>`.
- 03 Render props: `<Tooltip render={...}>`.
- 04 as-prop для polymorphism.
- 05 Forward ref – для интеграции с DOM API.
- 06 Controlled / uncontrolled – поддержка обеих форм.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дизайн-системы.
- 02 Библиотеки UI-компонентов.
- 03 Внутренние widget-collections.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много props → confusing.
- 02 Нет escape hatch → пользователю придётся форкать.
- 03 Только controlled / только uncontrolled – узкий API.
- 04 Compound без TypeScript-навигации – IDE не помогает.

КОД

// Compound-components

```
<Tabs defaultValue="a">
  <Tabs.List>
    <Tabs.Trigger value="a">A</Tabs.Trigger>
    <Tabs.Trigger value="b">B</Tabs.Trigger>
  </Tabs.List>
  <Tabs.Content value="a">...</Tabs.Content>
  <Tabs.Content value="b">...</Tabs.Content>
</Tabs>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем render-props лучше children?

02 Когда нужен Polymorphic компонент?

03 Что такое controlled и uncontrolled API?

ПОЛНАЯ СТАТЬЯ

<https://www.radix-ui.com/primitives/docs/overview/introduction>

Как избежать prop drilling?

ОПИСАНИЕ

Prop drilling – пробрасывание prop через несколько уровней без использования. Решения: Context (для редко меняющихся), композиция (передавать children, а не props), state-management (Zustand/Redux), компоненты-контейнеры, render-props.

КАК РАБОТАЕТ

- 01 Context – для глобальных значений (тема, юзер).
- 02 Композиция – поднять children в общего предка.
- 03 Slot-pattern: вместо props передать готовый JSX.
- 04 Store – для часто-меняющихся данных.
- 05 Hook – useUser() вместо передачи user через 5 уровней.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Когда видим >3 уровней пробрасывания одной prop.
- 02 При рефакторинге глубоких деревьев.
- 03 При выделении дизайн-системы.

ПОДВОДНЫЕ КАМНИ

- 01 Context – массовые ререндеры.
- 02 Композиция – иногда требует переосмысления структуры.
- 03 Глобальный store ради одного prop = окверкилл.
- 04 Каждое решение имеет свой trade-off.

КОД

// Композиция вместо drilling

```
// плохо
<Page user={user}><Header user={user} /></Page>

// хорошо – поднимаем разметку
<Page header={<Header user={user} />} />
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда лучше Context, когда композиция?
- 02 Что такое slot-pattern?
- 03 Чем хук useUser лучше пробрасывания?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/passing-data-deeply-with-context>

ВОПРОС Q30

Как избежать over-abstraction?

ОПИСАНИЕ

Принципы: rule of three (абстракция оправдана с третьего использования); not-DRY (иногда копипаста проще, чем неудачная абстракция); явные имена вместо generic; не «угадывать будущее». Преждевременная абстракция превращает код в неподдерживаемый. Правило обычно – "чуть-чуть мало абстракции лучше, чем чуть-чуть много".

КАК РАБОТАЕТ

- 01 Rule of three: первая копия – пиши; вторая – терпи; третья – абстрагируй.
- 02 Premature optimization is the root of all evil.
- 03 Имена должны отражать реальность, не «обобщённую будущую».
- 04 Generic-функция должна иметь >3 разных вызовов с разными типами.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review.
- 02 При появлении желания «переписать на красиво».
- 03 Когда новый член команды не понимает код.

ПОДВОДНЫЕ КАМНИ

- 01 5-этажные generic типы под «гибкость» которой никогда не будет.
- 02 Hook, который объединяет 5 разных useEffect под одним именем.
- 03 Wrapper-component, который ничего не добавляет.
- 04 Конфигурация через 30 props – сложнее, чем копипаста.

КОД

// Антипаттерн

```
// слишком обобщённое
function useGenericResource<T, K extends keyof T, ...>({
  resourceType: ResourceType<T, K, ...>,
  ...
}) { ... }

// лучше явно
function useUser(id: string) { ... }
function usePost(id: string) { ... }
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем DRY иногда вредит?
- 02 Когда копипаста – это ок?

03 Как обсуждать over-abstraction в команде?

ПОЛНАЯ СТАТЬЯ

[https://en.wikipedia.org/wiki/Rule_of_three_\(computer_programming\)](https://en.wikipedia.org/wiki/Rule_of_three_(computer_programming))

ВОПРОС Q31

Как проводить frontend architecture review?

ОПИСАНИЕ

Архитектурный ревью — раз в N месяцев или при крупных изменениях. Чек-лист: структура папок, нарушения слоёв, размеры файлов, дубли, старые `as` / `@ts-ignore`, performance-issues, security, accessibility. Цель — выявить проблемы и составить roadmap рефакторинга. Не наказание команды — обучение и улучшение.

КАК РАБОТАЕТ

- 01 Inventory: что есть, какие boundaries.
- 02 Метрики: bundle-size, LCP, FID, INP, % `any`.
- 03 Code health: lint warnings, complexity (madge / code-climate).
- 04 Security: dependency audit, secret leaks.
- 05 Documentation: ADR, README, Storybook coverage.
- 06 Roadmap: что чинить в следующий квартал.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Каждые 3-6 месяцев.
- 02 Перед большими релизами.
- 03 При onboarding нового tech lead.
- 04 После серьёзного incident.

ПОДВОДНЫЕ КАМНИ

- 01 Ревью без последствий = пустая трата времени.
- 02 Слишком много замечаний → команда демотивирована.
- 03 Без приоритизации → ничего не чинят.
- 04 Отчёт никто не читает.

КОД

// Шаблон отчёта

```
Architecture Review (Q3 2025)
Structure: 80% соответствует FSD
Issues:
  - features/* импорты entities/* в обратку (12 случаев)
  - shared/utils — 200+ экспортов (нужна разбивка)
  - 87 any в коде (target Q4: <40)
Roadmap:
  - Q4: разбить shared/utils
  - Q4: убрать as any в payment-flow
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие метрики смотреть?
- 02 Как приоритизировать issues?
- 03 Кто должен проводить ревью?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/agile-architecture.html>

В О П Р О С Q 3 2

Как документировать frontend architecture?

ОПИСАНИЕ

Минимум: README на корне (структура, как запустить), Architecture.md (схема слоёв, правила импорта), ADR (architecture decision records – почему выбрали X), Storybook (UI-каталог), TypeDoc / handcrafted docs для core APIs. Документация должна обновляться при изменениях – иначе деградирует.

КАК РАБОТАЕТ

- 01 README – entry point.
- 02 Architecture.md – схема, правила, ссылки на ADR.
- 03 ADR (Architecture Decision Records) – markdown в /docs/adr/.
- 04 Storybook – компоненты + interactive playground.
- 05 Code comments для сложных мест.
- 06 Mermaid / Excalidraw – диаграммы.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие проекты с долгим жизненным циклом.
- 02 Множественные команды.
- 03 Open source.
- 04 Onboarding новых разработчиков.

ПОДВОДНЫЕ КАМНИ

- 01 Документация без обновления – устаревает за пол года.
- 02 Слишком много документации никто не читает.
- 03 Storybook не поддерживается → перестаёт отражать реальность.
- 04 Решения без ADR → "почему так" забывается через год.

КОД

// ADR-шаблон

```
# ADR-007: Switch from Redux to Zustand
Status: Accepted (2025-09-12)
Context: Redux boilerplate замедляет команду
Decision: Используем Zustand для нового кода
Consequences:
  + меньше boilerplate
  + проще onboarding
  - две библиотеки одновременно во время миграции
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое ADR?
- 02 Чем Storybook полезен?
- 03 Как поддерживать документацию live?

ПОЛНАЯ СТАТЬЯ

<https://adr.github.io/>

ВОПРОС Q33

Как принять архитектурное решение?

ОПИСАНИЕ

Шаги: определить проблему, собрать context, сформулировать варианты (минимум 2-3), оценить trade-offs, обсудить в команде, выбрать, оформить ADR, имплементировать пилотом, ретроспектива. Решения не вечны – пересматриваются при изменении контекста.

КАК РАБОТАЕТ

- 01 Problem statement: что именно болит.
- 02 Options: минимум 2 варианта (включая «ничего не менять»).
- 03 Critique: pros/cons каждого.
- 04 Decision: обоснование выбора.
- 05 ADR: запись.
- 06 Pilot: первое внедрение в малом scope.
- 07 Retro: что вышло, нужны ли коррекции.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое решение, которое сложно отменить.
- 02 Выбор библиотеки.
- 03 Изменение архитектуры.
- 04 Технические компромиссы.

ПОДВОДНЫЕ КАМНИ

- 01 Решение без альтернатив – bias.
- 02 Нет ADR → через год никто не помнит.
- 03 Big-bang внедрение без пилота.
- 04 Игнорирование команды → саботаж.

КОД

// Скелет решения

```
Problem: Redux boilerplate замедляет.  
Options: 1. Зустанд 2. RTK 3. Не менять  
Decision: Zustand для нового кода  
Pilot: feature 'notifications' в Q4  
Retro: апрель 2026
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое decision matrix?
- 02 Как уговорить команду?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/architect-evolutionary-design.html>

ВОПРОС Q34

Когда писать ADR?

ОПИСАНИЕ

ADR (Architecture Decision Record) пишется на каждое значимое архитектурное решение: выбор библиотеки/стека, изменение слоёв, отказ от практики, новый паттерн. Не нужен для рутинных задач (выбор имени переменной). Хранится в `/docs/adr/`, имеет статус (Accepted, Rejected, Superseded), не редактируется после принятия (только новый ADR superseding).

КАК РАБОТАЕТ

- 01 Шаблон: Title, Status, Context, Decision, Consequences.
- 02 Нумерация: ADR-001, ADR-002, ...
- 03 Статусы: Proposed → Accepted → Superseded.
- 04 Хранение: `/docs/adr/ADR-NNN-title.md`.
- 05 Не редактировать – новый ADR supersedes старый.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Выбор state-management.
- 02 Switch роутера.
- 03 Новые архитектурные слои.
- 04 Отказ от устаревшей практики.
- 05 Принятие компромисса.

ПОДВОДНЫЕ КАМНИ

- 01 ADR на каждый pull request → шум.
- 02 ADR без context → бесполезен через год.
- 03 Редактирование старых ADR – теряется история.
- 04 Нет процесса аппрува → ADR без обсуждения.

КОД

// ADR-минимум

```
# ADR-012: Adopt FSD methodology
Status: Accepted (2026-01-15)
Context: Структура папок хаотична, новые
разработчики долго ориентируются.
Decision: Внедрить FSD пилотом в Q1.
Consequences:
+ единая структура
+ быстрый onboarding
- время на миграцию ~1 квартал
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем ADR отличается от RFC?
- 02 Кто аппрувит ADR?
- 03 Как пересматривать ADR?

ПОЛНАЯ СТАТЬЯ

<https://adr.github.io/>

ВОПРОС Q35

Как оценивать архитектурный trade-off?

ОПИСАНИЕ

Каждое решение – компромисс. Оси: производительность, поддерживаемость, гибкость, сложность, время имплементации, риск, обучаемость для команды. Нет идеальных решений – есть подходящие под контекст. Важно явно проговорить, ЧЕМ жертвуем – это снимает недопонимание позже.

КАК РАБОТАЕТ

- 01 Decision matrix: варианты × критерии.
- 02 Веса критериев – приоритеты команды/продукта.
- 03 Пилот / spike – пробный имплементейшн.
- 04 Reversibility: легко ли откатить.
- 05 Time-to-payoff: когда decision окупится.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое нетривиальное решение.
- 02 Сравнение библиотек.
- 03 Выбор паттерна.
- 04 Отказ от практики.

ПОДВОДНЫЕ КАМНИ

- 01 Игнорирование операционных costs.
- 02 Чрезмерный focus на performance в ущерб простоте.
- 03 Не учитывают обучаемость команды.
- 04 «Лучшее решение» в вакууме без контекста.

КОД

```
// Decision matrix
```

Criterion	Redux	Zustand	RTK
Boilerplate	x	✓	~
Ecosystem	✓	~	✓
Learning curve	x	✓	~
Server-state	x	~	✓ (RTK Query)
Bundle size	~	✓	~

МОГУТ ДОСПРОСИТЬ

- 01 Какие критерии добавлять в decision matrix?
- 02 Что такое reversibility?
- 03 Как обсуждать trade-offs в команде?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/architecture/>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- [] Описать структуру большого React-приложения и слои FSD (Q1, Q2, Q3, Q4)
- [] Сказать, где жить API-clients, hooks, domain services, UI components, design system (Q5, Q6, Q7, Q8, Q9)
- [] Объяснить, как отделять domain от UI и data fetching от render (Q10, Q11)
- [] Описать архитектуру таблицы, формы, wizard, dashboard (Q12, Q13, Q14, Q15)
- [] Описать loading / empty / error states (Q21)
- [] Расписать роли, permissions, защиту от unauthorized (Q16, Q17, Q18)
- [] Объяснить error boundaries – где и сколько уровней (Q19, Q20)
- [] Сказать, как делать graceful degradation, feature flags, gradual rollout (Q22, Q23, Q24)
- [] Описать i18n, theming, responsive (Q25, Q26, Q27)
- [] Описать shared component API, способы избегать prop drilling и over-abstraction (Q28, Q29, Q30)
- [] Объяснить процесс архитектурного ревью и документирования (Q31, Q32)
- [] Описать процесс принятия решений, ADR, оценку trade-offs (Q33, Q34, Q35)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru